



13.2 Student Guide: Microservices and Web Application Architecture

Class Overview

In the preceding class, we explored how the HTTP request and response communication framework relays information between a web server and client. We also looked at how web servers and clients use cookies to establish sessions.

Today's class will explore the web application infrastructure that makes the web client–server model possible. Specifically, we will cover:

- Why the original paradigm of monolith web application architecture evolved to the microservices model
- The different components of web applications
- Specific examples of web application components or "stacks"
- The deployment of web application architecture
- The database component of web applications and how to interact with data
- Using AI to create a website

Class Objectives

By the end of class, you will be able to:

- Understand how microservices and architecture work to deliver more robust, reliable, and repeatable infrastructure as code.
- Define the different services within a LEMP stack.
- Use GitHub Copilot to create a front end.
- Describe how relational databases store and retrieve data.
- Create SQL queries to view, enter, and delete data.

Notes

For the second half of the day, you will use Docker and Docker Compose to manage the activity's web application.

- `docker-compose up` and `docker-compose down` instantiate container environments specific to the directory they're run in.
-  **Head's Up:** Do not forget to run `docker-compose down` whenever you are done working in a directory where you ran `docker-compose up`.

Docker and Docker Compose Resources

The following are useful Docker and Docker Compose commands and resources:

- [Get started with Docker](#)
- [Overview of Docker Compose](#)
- [Remove a container volume](#)
- [Remove Docker Compose volumes](#)

To find running Docker-Compose container names and statuses:

- Navigate to where a `docker-compose.yml` file exists. For example:
 - `cd`
`~/Cybersecurity-Lesson-Plans/12-Web_Dev/deploying_testing_activity`
- Then run `docker-compose ps` to check the statuses:
 - `docker-compose ps` will return the containers associated with the compose file.
- Alternatively, use `docker container ls` to find all running containers on the system.

Directories and Files

To prepare for this class, confirm that the following files are present on your virtual machine:

- `/home/instructor/Cybersecurity-Lesson-Plans/12-Web_Dev/deploying_testing_demo/docker-compose.yml`

- `/home/sysadmin/Cybersecurity-Lesson-Plans/12-Web_Dev/deploying_testing_activity/docker-compose.yml`
- `/home/sysadmin/Cybersecurity-Lesson-Plans/12-Web_Dev/deploying_databases/docker-compose.yml`

If the above files are missing, you will need to update your VM.

Slides for this class can be found at [Slides 13.2](#)

01. Review Application Structure

On Day 1 of this module, we investigated and analyzed the HTTP request and response process, as well as sessions and cookies. These concepts are important for understanding web application communication and identity paradigms.

In today's class, we'll take a deeper look at the infrastructure behind web applications. Specifically, we'll look at:

- How modern web architecture has evolved to what is known as microservices
- What components make up microservices and their specific software "stacks"
- How web application components can be deployed using containers
- How databases work to understand how web applications store and transmit data
- How AI can be used to create websites easily using GitHub Copilot

Microservices Overview

To understand why microservices exist, we need to cover the following:

- How an application and its components are organized
- How information flows between components of an application
- How application architecture has changed from monoliths to microservices
- How to deconstruct a monolith into microservices

02 . Web Application Architecture

We will look at how an application is constructed to understand the paradigm shift in modern web architecture.

Consider this scenario:

- A developer, Andrew, uses a browser to manage a public-facing application (the Employee Directory App), where he retrieves and updates different kinds of employee lists for his company. This app is set up so that Andrew can add, remove, and view employees.

This application needs the following components to function:

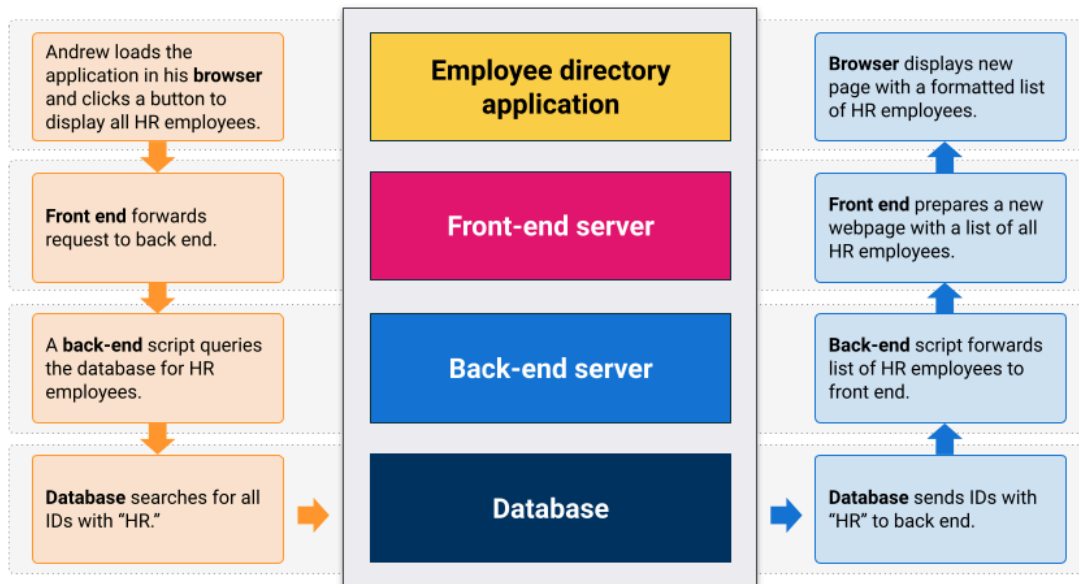
- A **front-end** server displays webpages and styles them in a readable format. This server is also responsible for receiving and responding to HTTP requests.
- A **back-end** server is responsible for executing business logic and writing or reading corresponding data to and from a database. The back-end server knows how to interact with the database depending on the specific request received.
 - For example, if Andrew requests to add a new employee, the back-end server knows to create a new number (e.g., 109) for the employee and then save that information in the database.
 - After this, the data is sent to the front end and formatted to display "New employee, Bob, created with employee ID, 109" in a new webpage.
- A **database** is used to store information on employees, such as their employee IDs and names. Critical and sensitive information is found in databases. We'll discuss this more in the second half of the class.

Information Flows Between Application Components

Now that we know the main components of a web application, we want to know what happens behind the scenes when a user interacts with a web application, or more specifically, makes an HTTP request to the server.

Let's find out how information flows between application components when Andrew requests a list of all employees in the HR department. The following image shows the process:

Information Flows Between Application Components



10

1. Andrew loads the application in his browser and clicks a button to see all HR employees.
2. The front end forwards the HTTP request to the back end.
3. A back-end script queries the database for HR employees.
4. The database searches for all IDs containing "HR."
5. The database sends IDs with "HR" to the back end.
6. A back-end script forwards the list of HR employees to the front end.
7. The front end prepares a new webpage listing all the HR employees.
8. The browser displays a new page to Andrew, with a formatted list of HR employees.

Each component of the web application serves one primary function, whether it's serving website data over HTTP or querying the database.

Modern Architecture

Modern web applications are composed of many microservices. Microservices are several services that run on a server in order to run a web application.

One of the first steps to creating microservices is separating the components into individual machines so that each one can run a "service."

A service, such as the front-end server from our earlier examples, is really just an operating system, running a minimum amount of software that has been configured to serve one main purpose, such as handling HTTP requests and responses.

This means that each individual machine requires:

- An operating system to run on, in order to run the service's software and store its configurations
- The software itself, which runs a specific set of instructions for that individual component of the web application
 - For example, a database service may run Linux as its operating system and have Microsoft's SQL Server installed on it.

Now we can explore what open-source and commercial software we need to set up to create our web application components.

Stacks

Any combination of a base operating system, front-end software, back-end software, and database is called a web **stack**.

Which stack you run depends on the needs of your organization. For example, let's look at the well-known LAMP stack. This stack consists of:

- Linux as the base operating system
- Apache to handle the front-end HTTP requests and responses
- MySQL for storing data
- PHP for back-end transformation of front-end requests and responses to database queries (we'll look at database queries in our last activity)

The LAMP stack is considered the foundational web stack, with a long history of use in architecting web applications. For example, the following sites either still use or originally used the LAMP stack:

- WordPress
- Facebook
- Tumblr
- Wikipedia

- Slack

 Visit the following link if you are interested in learning more about the history of the LAMP stack:

- [Tedium: The LAMP stack](#)

These days, there are many more web stacks that use different software. Note that the stack is simply a description of the software running in the web application.

- You can technically have a monolith or microservices web application using the same software stack.
- The difference is that monoliths have one operating system with all the software on it, while microservices have many smaller operating systems, each with the minimum amount of software needed for its own service.

But what stack do we want to use for our web application?

Introduction to Common Web Architecture

Let's start by comparing the different operating systems available for our services.

- The [Alpine Linux](#) is one example. It is a super lightweight Linux distribution with the minimum requirements to run containers.
- Other container-friendly operating systems include most other Linux distributions, such as Ubuntu, and more recent versions of Windows Server. The following are base operating systems used for web applications:
 - Alpine Linux
 - Common Linux distros such as Ubuntu, Red Hat, and SUSE
 - Windows Server 2016 and higher

We also have the front-end server that handles the HTTP requests and responses. This is usually some form of HTTP server running on a lightweight containerized operating system.

The front-end server is the part of a web application that renders the visual components of a web application. HTML, CSS, and JavaScript are the primary front-end web design languages.

- The following are specific examples of front-end servers:

- Apache
- nginx (pronounced "engine X")
- IIS

Data for the application is housed in the database server. Remember that there could be multiple databases, some for storing sessions and cookies and some for storing business logic like a company's employee directory.

- The following are specific examples of databases:
 - MySQL
 - Note that you may also see MariaDB, which we will use in our activity.
 - MariaDB is a community version of MySQL, created after Oracle acquired MySQL. MariaDB tries to be as close to MySQL as possible but has an open-source license.
 - PostgreSQL
 - Microsoft SQL Server
 - CouchDB
 - MongoDB

Lastly, you usually need a back-end server that transmits data between the front end and the database. These servers run fully featured scripting and programming languages.

- The following are specific examples of back-end servers and languages:
 - PHP
 - Perl
 - Python
 - nodeJS

The WIMP Stack

There is also a Microsoft-based variation called the WIMP stack that uses the following:

- Windows operating system
- Microsoft's IIS web server in place of Apache and nginx
- Database, such as Microsoft SQL Server
- Back-end language, such as PowerShell

We won't be using WIMP, but it is important to be aware of Microsoft-based web stacks.

-  Review the following Wikipedia article for a brief overview of the WIMP stack:
 - [Wikipedia: WIMP stacks](#)

The web software stack that we'll be using for today's demos and activities consists of Linux, nginx, MariaDB, and PHP.

This is known as the **LEMP stack**.

The LEMP Stack

What goes into the LEMP stack?

- Linux: This is the standard underlying operating system used for web applications. However, we will be using it with containers.
- nginx: We'll be using nginx for our front-end HTTP server instead of Apache, mainly for its speed. While Apache has more features and is more compatible with back-end languages, such as Java, nginx is well-known for its performance.
- MariaDB: Unofficially the "other" MySQL, it is a standard relational database that will handle our data storage.
- PHP: Our pages will be generated with data pulled by PHP scripts. We will not need to manage or work with any PHP directly today, but know that it is acting as our back-end language.

This set of Linux, nginx, MariaDB, and PHP is a variation of the LAMP stack we talked about earlier.

We just covered all the software that we want to use for our web application. But what about the other side of web application architecture, the hardware?

- We need a hardware environment suitable for web application microservices that is lightweight and easy to deploy.
- For that, we can use virtualization. For the purpose of microservices, we need the most lightweight virtual environment possible.

Microservices Infrastructure with Containers

Microservices require lightweight environments because:

- Our microservices need to be rapidly deployable.
- We need to be able to scale our microservices to meet growing demand.
- Developers and maintainers need to be able to replicate their own copies of these microservices locally.
- Having full-sized virtual machines for each service would require more resources and be more costly.

? Can you think of a technology that can create isolated virtual environments?

- Lightweight virtualization environments exist in the form of **containers**.
 - Containers run as isolated, virtual operating systems that dynamically allocate resources depending on the container's needs. These are unlike regular virtual machines, where all the hardware is virtualized.
 - Docker is the most popular container platform. For the rest of this lesson, we will use Docker's implementations of containers.

Using containers is the primary way microservices are developed and will be the method we use to "miniaturize" the components of our web stack.

How a Container Becomes a Microservice

Containerization is the process of packaging all the requirements to set up a microservice as a container.

Containerizing a microservice does the following:

- Declares a base operating system for the microservice to run on
- Stages the microservice's source code to the container
- Sets a command that launches the microservice and all other required software when the container is deployed

The containerization process is declared in a simple text file called a Dockerfile.

- Dockerfiles contain all the configuration needed for a container in one file, similar to how an Ansible configuration file contains everything it needs to configure a host when you run the command `ansible-playbook your-playbook.yml`.

In the case of the Amazon Product microservices that we saw before, we would create a separate Dockerfile for each of the microservice components.

- Review the following sample of a Dockerfile:

```
#####

# Product Directory Application

#####

# Required Base OS

FROM ubuntu:14.04

...[truncated]
```

- In this Dockerfile, Docker will install an Ubuntu 14.04 image to serve as a base operating system for the microservice.
- Other declarations in the Dockerfile set up the software and configuration required for the microservice.
 - This varies depending on the service.

A unique Dockerfile will be created for each microservice.

You don't need to know how to construct or maintain a Dockerfile, but you should understand that these Dockerfiles can be repeatedly used to create and configure the same microservice set, similar to what Ansible playbooks do for Ansible.

03. Intro to Databases

 **Head's Up:** This demo, activity, and review contain important concepts that we'll build on in the upcoming Web Vulnerability module.

Let's recap everything we've learned so far today:

- How to separate a monolith into microservices
- How to describe web application architecture
- Defining specific web application architectures

- How to deploy and test that a container set is running and functional

Now that we have easily deployable microservice-based web applications, we will cover one of the most critical components of a web application: The database.

Databases are critical to web application architecture because they contain the persistent data that flows through the web application. The data in the database can include, but isn't limited to:

- The usernames, telephone numbers, and email addresses of customers
- The session and cookie IDs of site visitors

Even more critically, a database can have personally identifiable information (PII), such as the full names, dates of birth, credit card numbers, and social security numbers of customers.

As such, it is absolutely crucial that security professionals understand how databases work in web applications.

Note that our understanding of databases will be the foundation of our exploration of web vulnerabilities in an upcoming module.

Databases Overview

- Databases are used to store large amounts of data.
- Databases are typically kept separate from the other components of a web application.
 - This is for reasons related to both security (compromise of one machine does not imply compromise of the other) and scale (many servers can use the same database).

As we saw in our last demo and activity, this was the case in our `docker-compose.yml` file. We had multiple defined services, and the database was its own container `demo-db` in the demo and `db` in the activity.

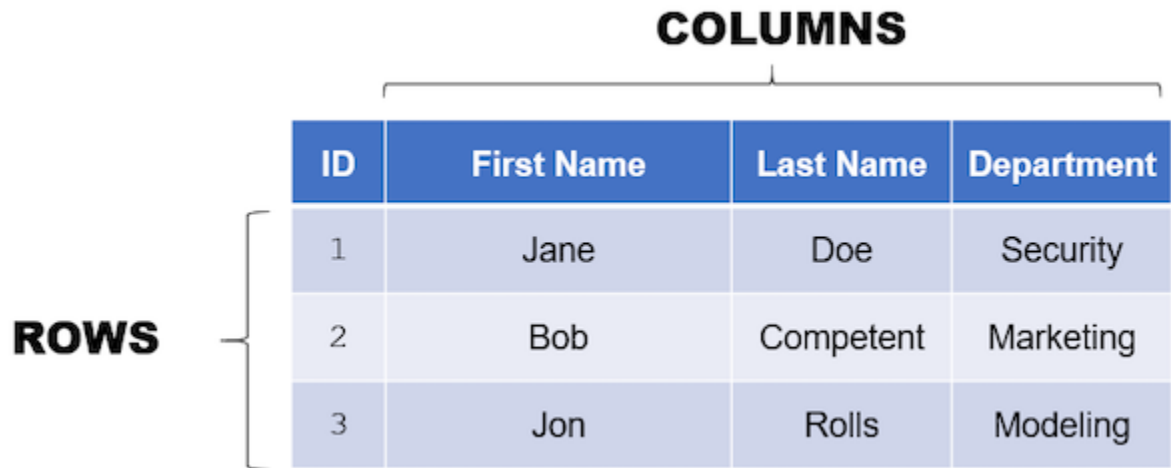
Database Types

There are many kinds of databases, but the most important for our purposes are SQL and NoSQL databases.

- We won't discuss NoSQL databases in class, but note that the techniques covered today only work for SQL databases.

SQL databases organize data like a spreadsheet:

- Each row is an item in the database and each column is a piece of data in the row.
- A whole "spreadsheet" of rows and columns is called a **table**.
- A collection of tables is called a database.



The diagram shows a table with four columns and three rows. The columns are labeled 'ID', 'First Name', 'Last Name', and 'Department'. The rows are labeled with IDs 1, 2, and 3. The first row contains 'Jane Doe' in the 'First Name' and 'Last Name' columns, and 'Security' in the 'Department' column. The second row contains 'Bob' in the 'First Name' column, 'Competent' in the 'Last Name' column, and 'Marketing' in the 'Department' column. The third row contains 'Jon' in the 'First Name' column, 'Rolls' in the 'Last Name' column, and 'Modeling' in the 'Department' column. A bracket on the left side of the table is labeled 'ROWS', and a bracket on the top side is labeled 'COLUMNS'.

ID	First Name	Last Name	Department
1	Jane	Doe	Security
2	Bob	Competent	Marketing
3	Jon	Rolls	Modeling

Once you've created a table, there are four main ways to interact with the data it contains:

- **Create** or add new entries. When a user makes a new account in a web app, they create a new database entry with their user info.
- **Read** or view entries. When a web application verifies your login credentials, it is reading the saved credentials in the database.
- **Update** or modify existing data. Whenever you need to reset your password in a web application, the password entry (more specifically, the hashed password) is updated with the new password.
- **Delete** or destroy data. If you have the option to delete your account from a web application (e.g., on a GDPR web app), your account info is deleted from the database.

Together, these are as known as **CRUD operations**. These operations are also known as **queries**.

- We'll use the term query to describe any input that interacts with data.

We'll discuss the SQL syntax for these operations to set the stage for studying **SQL injections**.

- SQL injections are a type of web application attack that manipulate queries so that an attacker can use a database in ways the developers did not originally intend.

Database Demo

Now that we understand the high-level concepts of database operations, let's create and enter queries into a web application's database to carry out basic database administration tasks.

We'll do the following database administration tasks in this demo:

- Deploy a web application.
- Navigate to the web application's page to see what data can be viewed by a web client.
- Start a bash session in the database's container and log into the database.
- Use a SQL query to retrieve the contents of the database's primary table.
- Modify that SQL query to find specific data.
- Add new data to the database.
- Delete data from the database.

1. First, navigate to our application's container set directory and stand it up.

- Run the following in a terminal:

1. `cd`

`/home/instructor/Cybersecurity-Lesson-Plans
/12-Web_Dev/deploying_databases`

- In this directory is a script called `reset_databases.sh` that will set up the database for this demo. The script will also reset the database to its original state if, at any point, the wrong SQL query is used.
- Run the script with `./reset_databases.sh`.
- Then, bring up the app with `docker-compose up`.

2. We should be able to see our deployed web app now:

- Open Firefox and navigate to `127.0.0.1:10005` to show the GoodCorp Employee Directory page.
- Scroll down the page and note that the employee entries are being read from a database.

3. Open a new terminal window and enter an interactive bash session in the container:

- Run `docker exec -it activitydb bash`. The prompt should change to `root@<containerID>`.
 - We should be familiar with how to enter an interactive bash session in a Docker container.
4. Enter a MySQL session like earlier, but this time, add the `-D` argument to specify a database to use:
- Run `mysql -u admin -p123456 -D goodcorpdb`.
 - The prompt will change to `MariaDB [goodcorpdb]`.
 - Of course, you should never use such an obvious username and password combination.
5. **SELECT queries:** In this database, we have a table called `employees` that has the list of employees we saw on the webpage earlier. To find all the entries in this table, we can use a `SELECT` statement to create a query that returns all the entries from the `employees` table.
- While in the MySQL session, run `SELECT * FROM employees;`
6.  **Head's Up:** Always remember to end your queries with a semicolon (`;`) or your database will expect more SQL.

Run `SELECT * FROM employees;` to show all the employees we saw earlier:

```
MariaDB [goodcorpdb]> SELECT * FROM employees;
```

```
+----+-----+-----+-----+-----+-----+
| id | firstname | lastname | email | department | date_added |
+----+-----+-----+-----+-----+-----+
| 1 | Bob | Brew | bbrew@goodcorp.net | Sales and Marketing | 2020-09-16
11:00:44 |
| 2 | Andrew | Americano | aamericano@goodcorp.net | Research and Development |
2020-09-16 11:00:44 |
| 3 | Caroline | Cortado | ccortado@goodcorp.net | Human Resources | 2020-09-16
11:00:44 |
```

```
| 4 | Deborah | Doppio | ddoppio@goodcorp.net | Operations | 2020-09-16  
11:00:44 |
```

```
| 5 | Emma | Espresso | eespresso@goodcorp.net | Research and Development |  
2020-09-16 11:00:44 |
```

```
+---+-----+-----+-----+-----+-----+
```

- 5 rows in set (0.000 sec)
 - We can see that we have five employees in this table and that they are organized by their employee **ID**, **first name**, **last name**, **department**, and the **date** and **time** they were added to the database.
7. **WHERE clause:** The **WHERE** clause allows us to narrow down results when searching through a database. We can append the **WHERE** clause to a query to narrow down and modify what it's doing. **WHERE** acts as a conditional statement that returns database results when it is met.
- In this example, we'll use **WHERE** to find all employees in the Operations department. To do so, we specify that when we use **SELECT * FROM employees**, we only want the rows where the **department** is equal to **'Operations'**:

Run **SELECT * FROM employees WHERE department='Operations'**;

```
MariaDB [goodcorpdb]> SELECT * FROM employees WHERE department='Operations'  
  
-> ;
```

```
+---+-----+-----+-----+-----+-----+
```

```
| id | firstname | lastname | email | department | date_added |
```

```
+---+-----+-----+-----+-----+-----+
```

```
| 4 | Deborah | Doppio | ddoppio@goodcorp.net | Operations | 2020-09-16 11:00:44 |
```

```
+---+-----+-----+-----+-----+-----+
```

- 1 row in set (0.001 sec)

- Using **WHERE** to modify a SQL query is necessary when dealing with databases with huge amounts of data.

8. **INSERT INTO queries:** If we want to add a new employee to our database, we use the **INSERT INTO** query.

- This query adds new data to a database and takes the format of:

```
INSERT INTO table_name (column1, column2, column3, ...)
```

9. **VALUES** (value1, value2, value3, ...);

Construct a query using this format:

```
INSERT INTO employees (firstname, lastname, email, department)
```

- **VALUES** ('Luke', 'Latte', 'llatte@goodcorp.net', 'Logistics');
- We are not entering column values for **Id** or **date_added** because they are automatically incremented when we enter a new row.
- Run the query to show the **Query OK, 1 row affected (0.001 sec)** output, confirming a successful **INSERT** query.

Check the entire table again with **SELECT * FROM employees;** to see our new employee, Luke, at the bottom of the table:

```
MariaDB [goodcorpdb]> SELECT * FROM employees;
```

```
+----+-----+-----+-----+-----+-----+
| id | firstname | lastname | email | department | date_added |
+----+-----+-----+-----+-----+-----+
| 1 | Bob | Brew | bbrew@goodcorp.net | Sales and Marketing | 2020-09-16 11:00:44 |
| 2 | Andrew | Americano | aamericano@goodcorp.net | Research and Development | 2020-09-16 11:00:44 |
```

| 3 | Caroline | Cortado | ccortado@goodcorp.net | Human Resources | 2020-09-16 11:00:44 |

| 4 | Deborah | Doppio | ddoppio@goodcorp.net | Operations | 2020-09-16 11:00:44 |

| 5 | Emma | Espresso | eespresso@goodcorp.net | Research and Development | 2020-09-16 11:00:44 |

| 6 | Luke | Latte | llatte@goodcorp.net | Logistics | 2020-09-16 11:29:00 |

+---+-----+-----+-----+-----+-----+-----+

- 6 rows in set (0.000 sec)
- Return to Firefox and refresh the page to see how it shows up on the employee directory page:
 - Go back to Firefox and refresh the page to see the new employee listed at the bottom.

10. **DELETE queries:** To delete the entry we just created, we use a **DELETE** query. We want to use this together with the **WHERE** clause to specify deleting only a single entry.

- Run **DELETE FROM employees WHERE email='llatte@goodcorp.net';** and note the **Query OK** output.
-  **Head's Up:** You never want to enter a query like **DELETE FROM employees;** as it will delete the entire **employees** table!
 - If attackers gain access to a database, they could use **DELETE** to maliciously wipe out thousands or even millions of records, significantly affecting a business's operations.
- Let's return to using **SELECT** to verify that our deleted user is no longer listed in the database:

SELECT * FROM employees; to show that we're back down to five employees:

MariaDB [goodcorpdb]> **SELECT * FROM employees;**

+---+-----+-----+-----+-----+-----+-----+

| id | firstname | lastname | email | department | date_added |

```

+---+-----+-----+-----+-----+-----+
| 1 | Bob      | Brew      | bbrew@goodcorp.net | Sales and Marketing | 2020-09-16 |
| 11:00:44 |
| 2 | Andrew    | Americano | aamericano@goodcorp.net | Research and Development |
| 2020-09-16 11:00:44 |
| 3 | Caroline  | Cortado   | ccortado@goodcorp.net | Human Resources      | 2020-09-16 |
| 11:00:44 |
| 4 | Deborah   | Doppio    | ddoppio@goodcorp.net | Operations            | 2020-09-16 |
| 11:00:44 |
| 5 | Emma      | Espresso  | eespresso@goodcorp.net | Research and Development |
| 2020-09-16 11:00:44 |
+---+-----+-----+-----+-----+-----+

```

- 5 rows in set (0.000 sec)
- For visual confirmation via the employee directory, reload the webpage to see that the entry is now gone.

04. Activity: Database Management

- [Activity File: Database Management](#)
-

05. Break

06. Segue in SQL Injections

Before ending our lesson on databases, we'll look at some queries that may seem pointless or improperly created but are actually fundamental to how SQL injections work.

- `SELECT * FROM employees WHERE 1=1;`

? Do you know what will happen when this query is run?

- When we run this query, the `1=1` part of the statement evaluates to true since the number one always equals one, therefore, running the `SELECT * FROM employees` statement.
- Run the query to see that it retrieves the entire `employees` table again.

This seems like a pointless condition for the `WHERE` clause because we can simply use `SELECT * FROM employees`. However, creating a statement that results in being true is one way that SQL injections work.

? During the previous activity, did you forget to add the semicolon to the end of a query?

- Type `SELECT * FROM employees` without a semicolon and press Enter a few times.

It does not do anything. The database is still waiting for more SQL.

MariaDB [goodcorpdb]> `SELECT * FROM employees`

->

->

->

○ ->

This expectation of explicit semicolon endings can be leveraged in SQL injections.

- We'll go into much more depth on exploiting databases in an upcoming module, which will focus on the offensive side of security and build directly on this module's lessons.

 For more information on offensive security definitions of ethical hacking and penetration testing, review the following resources:

- [Synopsis: Ethical hacking](#)
- [Rapid7: Penetration testing](#)

07. Using AI to create a web application

We will now use GitHub Copilot, an AI application, to craft a web application. This application will not require any microservices or a database, as it will just be a front end.

Setup

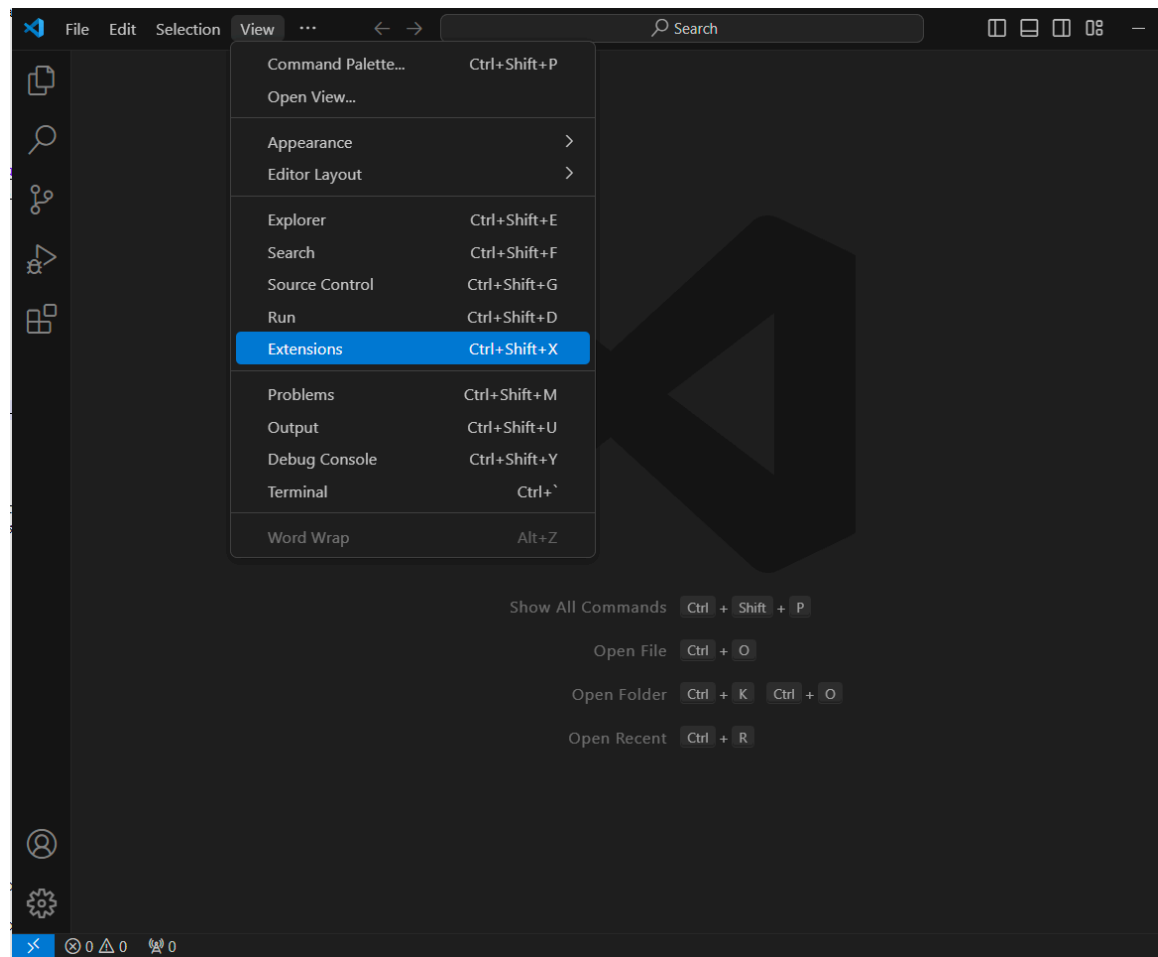
Setting up GitHub Copilot involves a few steps to get everything running.

1. Prerequisites:

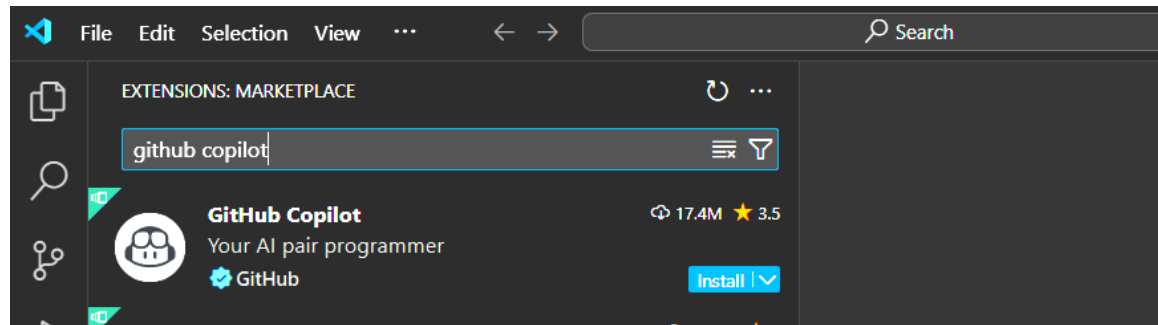
- **GitHub account:** You will need to have a GitHub account. You can sign up for free at [GitHub](https://github.com).
- **Visual Studio Code:** You will also need to download and install [Visual Studio Code \(VS Code\)](https://code.visualstudio.com/).

2. Install GitHub Copilot extension:

- **Open VS Code:** Launch Visual Studio Code on your computer.
- **Extension marketplace:** Click on **Extensions** from the **View** drop-down menu (or press **Ctrl+Shift+X**).



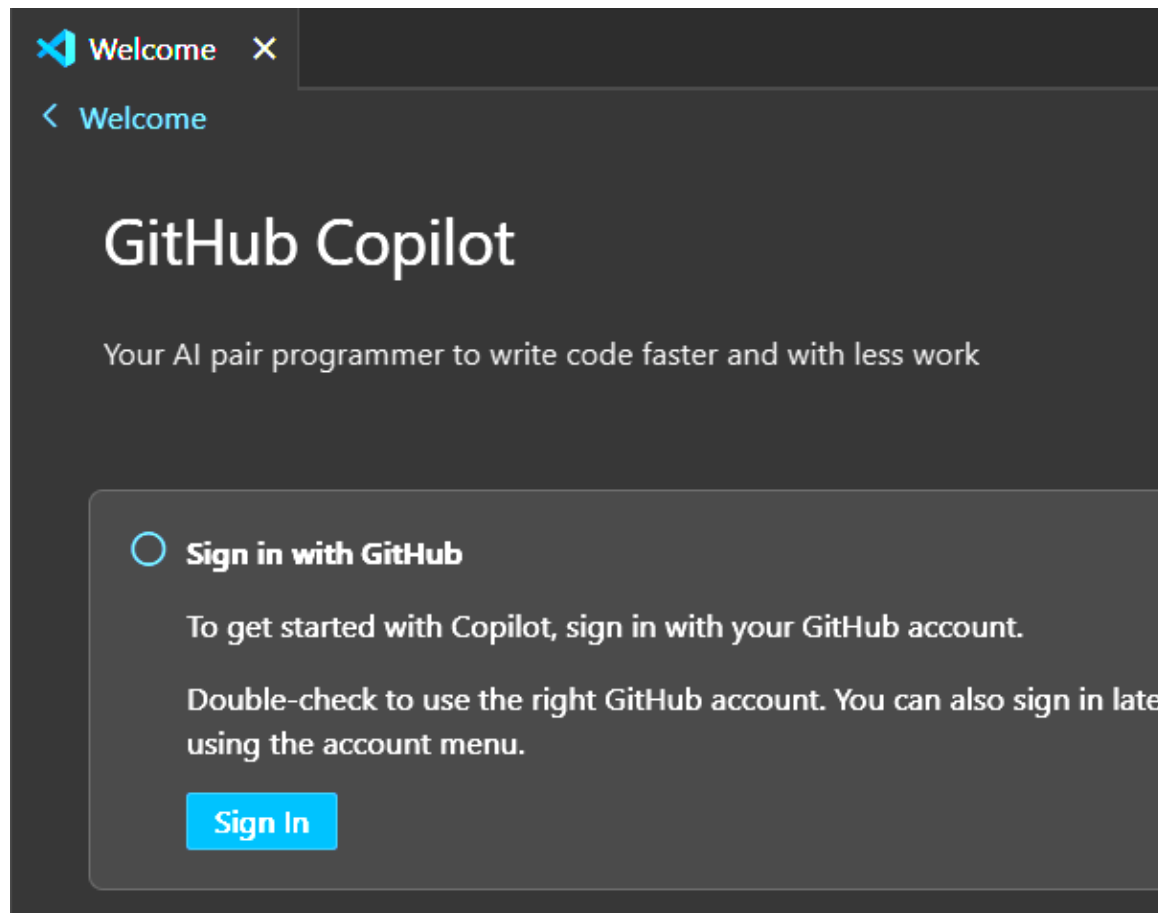
- **Search for GitHub Copilot:** In the **Extensions** view, type "GitHub Copilot" in the search bar.



- **Install the extension:** Find the GitHub Copilot extension in the search results and click the **Install** button.

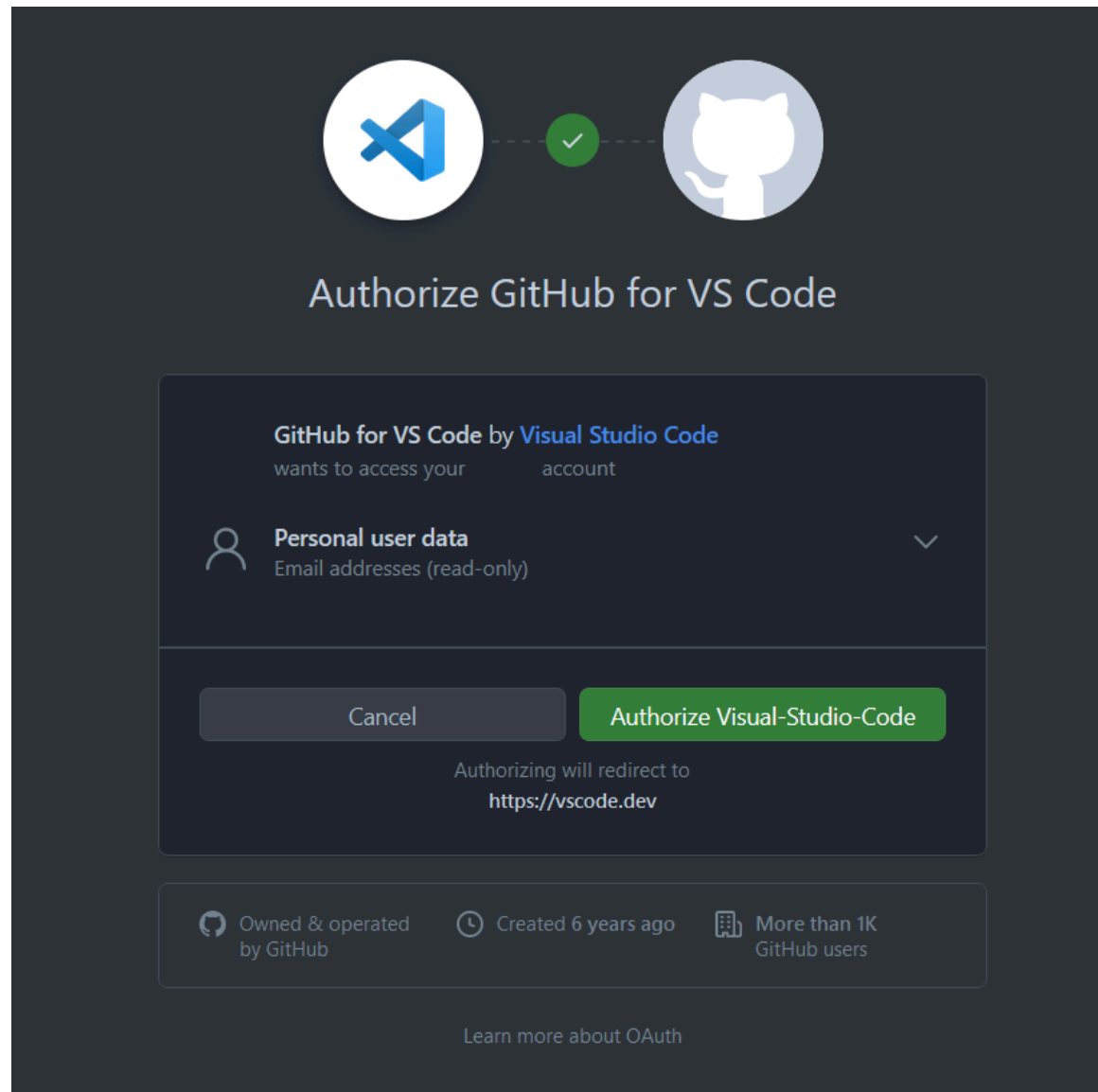
3. Sign in to GitHub:

- **Activate extension:** After installing, you will see a prompt to sign in to GitHub to activate GitHub Copilot. Click on **Sign In**.

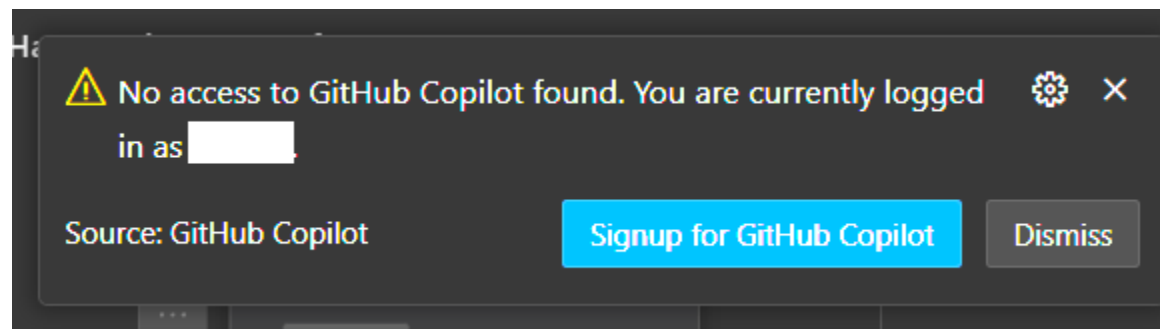


- **Authentication:** This will open a browser window where you can authenticate with your GitHub account. Follow the instructions to

complete the sign-in process.



- **CoPilot signup:** You will see a message now pop up to sign up for Copilot. Click **Signup for GitHub Copilot** and then click **Get access to GitHub Copilot**.



- **Allow public code:** When prompted, change the option for 'Suggestions matching public code (duplication detection filter)' to 'Allowed'.

You can change these at any time from your account settings.

ⓘ Select an option below to continue.

Copilot policies

Copilot in the CLI You can use GitHub Copilot for assistance in terminal.	Enabled
Copilot Chat in the IDE You can use GitHub Copilot Chat in the IDE to explain a piece of code, propose bug fixes, or generate unit tests in a chat interface from your editor.	Enabled
Copilot Chat in GitHub Mobile You can use Copilot Chat in GitHub Mobile , personalized to a codebase.	Enabled
Suggestions matching public code (duplication detection filter) Copilot can allow or block suggestions matching public code. Learn more about code suggestions .	Allowed ▾
☒ Allow GitHub to use my code snippets from the code editor for product improvements * Allow GitHub, its affiliates and third parties to use my code snippets to research and improve GitHub Copilot suggestions, related models and product features. More information in About GitHub Copilot privacy	

For more information about the data your organization receives regarding your use of GitHub Copilot, please review [GitHub's Privacy Statement](#).

Save and complete setup

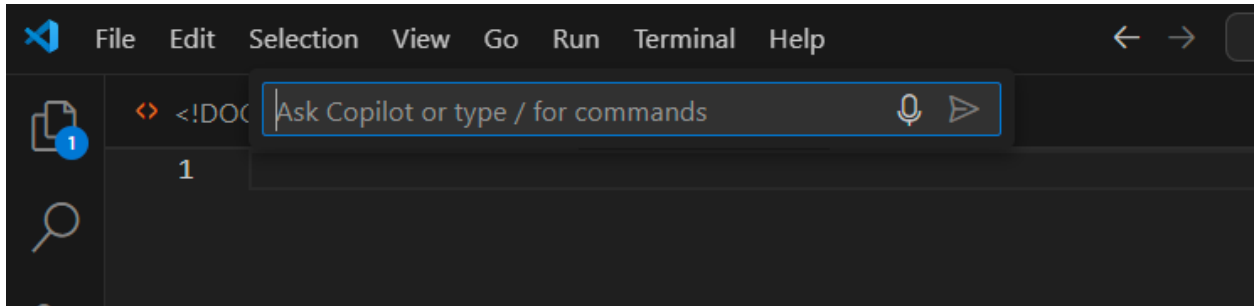
4. **Note:** Click **Sign-in** again in the bottom window if it doesn't automatically sign you in.

Using GitHub Copilot

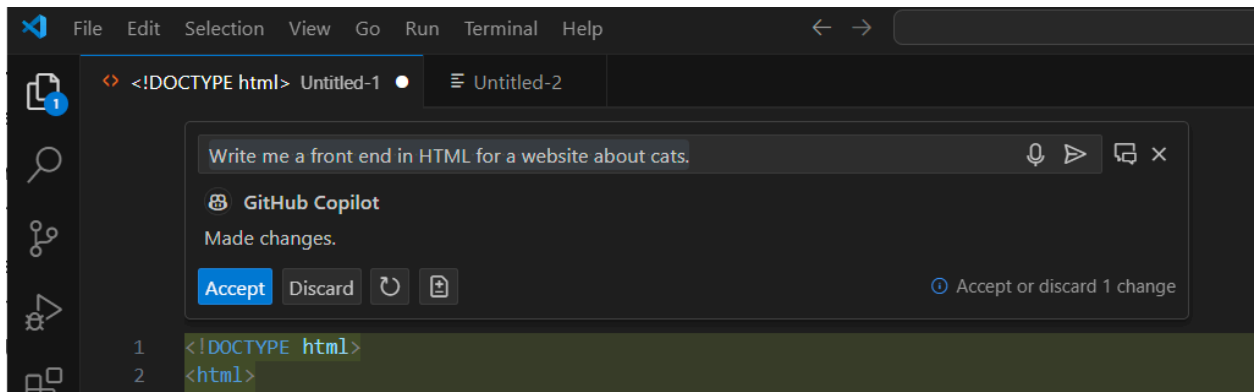
We will now use Copilot to develop a front end.

1. Open a new file by going to File > New Text File.

2. In the text file window, press CTRL+I to open a chat prompt with Copilot. **Note:** If the prompt doesn't appear or let you enter a prompt, restart VS code.



3. In the prompt, ask it to create an HTML website on a topic of your choice.



4. Save the file to your machine (make sure it ends in .html) and open it in a browser by double-clicking on it.

This is a very basic front end of a website that is a standalone file. It doesn't use any SQL databases or backend services. Copilot can't stand up infrastructure that is required for those types of setups, however, it is very useful for making standalone pages or generating code in general.

09. Activity: Creating a website with Copilot (0:20)

- [Activity File: Creating a website with Copilot](#)